

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Multidisciplinary Minor

Course code: MDC401

Year: SE SEM: IV

Experiment No. 9
AIM: Building a Simple Counter App using React Hooks. Use React Hooks (useState, useEffect) to manage state and side effects.
Name: Prem Panchal
Roll Number: 48
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty: Dr. Yogesh Pingle

Aim: Building a Simple Counter App using React Hooks. Use React Hooks (useState, useEffect) to manage state and side effects.

Code:

▪ Counter.js

```
import React, { useState, useEffect } from 'react';

// This is a Functional Component. It accepts "props" (properties) from its parent.
function Counter(props) {

  // 1. useState Hook:
  // We declare a state variable named 'count' and a function to update it 'setCount'.
  // We initialize it using a prop passed from the parent.
  const [count, setCount] = useState(props.initialCount || 0);

  // 2. useEffect Hook:
  // This runs after every render, BUT the dependency array [count] tells it
  // to ONLY run when the 'count' variable actually changes.
  useEffect(() => {
    // Side effect: Update the document title (the text on your browser tab)
    document.title = `Clicked ${count} times`;
    console.log(`The counter changed to: ${count}`);
  }, [count]);

  return (
    <div style={styles.card}>
      /* Displaying the prop passed from the parent */
      <h2>{props.title}</h2>

      /* Displaying our state variable */
      <p style={{ fontSize: '32px', fontWeight: 'bold' }}>{count}</p>

      /* Buttons that trigger the setCount state updater */
      <button style={styles.btnGreen} onClick={() => setCount(count + 1)}>
        Increment
      </button>

      <button style={styles.btnRed} onClick={() => setCount(count - 1)}>
        Decrement
      </button>
      <br /><br />
      <button style={styles.btnGray} onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}
```

```

    </div>
  );
}

// Just some basic inline CSS styling for our component
const styles = {
  card: { padding: '20px', border: '1px solid #ccc', borderRadius: '8px', maxWidth: '300px',
margin: '20px auto', textAlign: 'center', boxShadow: '0 4px 8px rgba(0,0,0,0.1)' },
  btnGreen: { padding: '10px 15px', margin: '5px', cursor: 'pointer', backgroundColor:
'#28a745', color: 'white', border: 'none', borderRadius: '4px' },
  btnRed: { padding: '10px 15px', margin: '5px', cursor: 'pointer', backgroundColor:
'#dc3545', color: 'white', border: 'none', borderRadius: '4px' },
  btnGray: { padding: '8px 12px', margin: '5px', cursor: 'pointer', backgroundColor:
'#6c757d', color: 'white', border: 'none', borderRadius: '4px' }
};

export default Counter;

```

▪ App.js

```

import React from 'react';
import Counter from './Counter'; // Import our new component

function App() {
  return (
    <div style={{ fontFamily: 'Arial', textAlign: 'center', marginTop: '50px' }}>
      <h1>Experiment 9: ReactJS Hooks</h1>

      {/* Here we render the Counter component.
       We are passing two PROPS to it: 'title' and 'initialCount'.
       */}
      <Counter title="My Custom Counter" initialCount={0} />

    </div>
  );
}

export default App;

```

Output:

